

Swift: 基于强化学习的多信号融合自适应网络拥塞控制算法

杭天铨

南京邮电大学

1224045520@njupt.edu.cn

摘要

现代网络传输正在从单一固定终端和局域高质量链路,扩展到跨地域远距离传输、蜂窝/无线/卫星等多接入网络,以及手机、笔记本、台式机和边缘设备等多模态终端并存的复杂场景。传统基于丢包的算法存在拥塞信号滞后与缓冲区膨胀问题,基于延迟的算法则面临 RTT 测量噪声与共存公平性不足等缺陷;在长 RTT、链路抖动和终端能力差异并存时,静态参数和单一信号驱动的拥塞控制更难兼顾吞吐、延迟与稳定性。本文提出 Swift 算法——一种基于强化学习辅助的多信号融合自适应网络拥塞控制方法。其核心创新收敛为三点:(1)构建 11 维有效 RL 观测子空间嵌入于 15 元素 OpenGym 传输容器(另含套接字标识、环境类型、仿真时间、节点标识四项元数据通道)中,将 ECN 状态、拥塞控制状态机状态、事件语义、RTT 和在途字节等协议栈内部信息纳入强化学习状态表示,以提升对远距离和异构终端传输状态的可观性;(2)提出多信号融合拥塞判定与差异化响应机制,协同分析丢包、超时与 ECN 信号,并按信号严重程度施加差异化窗口保留因子(丢包 $\rho = 0.70$ 、ECN $\rho = 0.75$ 、超时 $\rho = 0.50$),同时通过连续递减保护($D_{\max} = 3$)和降窗后冻结机制抑制窗口过度收缩与快速反弹;(3)设计强化学习辅助的自适应融合控制策略,基于最小 RTT 感知的 RTT 膨胀阈值与奖励信号指数移动平均动态调整乘性增加因子 $\alpha \in [0.85, 1.30]$,并融合 BDP 窗口化最大值滤波估计、目标窗口渐进逼近、HyStart 风格慢启动退出和管道利用率感知加权递增。在 ns-3 离散事件仿真平台覆盖近端高速、无线接入、5G、城域/广域网与卫星通信的 21 个场景、84 组实验中,经数据质量审计筛选出 9 个具备完整四协议对照的主要场景。稳态性能(纯 TCP 设置)结果表明:Swift 在近端高速场景中与新 Reno/CUBIC 保持等价吞吐量(差距 $< 0.01\%$),同时显著压低排队延迟;在 5G、城域 WAN、WiFi 等中速接入场景中维持零丢包并降低多数场景延迟;在 GEO 卫星长距离链路中凭借零丢包特性相对 NewReno/CUBIC 均值实现吞吐量提升 28.8%。跨流量鲁棒性(800 Mbps UDP Burst 对照设置)进一步表明:在全部 17 个完整对照场景中,Swift 是唯一维持零丢包率的算法,且吞吐量回落幅度最小。本文更侧重从拥塞信号时序、队列驻留、带宽估计稳定性和长 RTT 恢复代价等角度进行定性比较,仅在关键场景给出可追溯的定量结果;同时披露 Swift 在少数中低速与中等 RTT 场景下存在小幅度延迟抬升,以及部分广域网场景下吞吐量存在小幅回落,其机理为 $\alpha \times BDP$ 目标窗口在高 RTT 环境下的扩张幅度与 BDP 估计仍有进一步优化空间,由此为面向排队延迟感知的 α 动态调节与多时间尺度 BDP 估计指明了进一步方向。

关键词: 拥塞控制; 强化学习; 多信号融合; 显式拥塞通知; 自适应参数调优; 远距离传输; 多模态终端

1 引言

随着云计算、大数据、人工智能、移动互联网和物联网技术的蓬勃发展,端到端网络传输的服务对象正在从固定主机扩展到手机、笔记本、台式机、边缘节点和传感设备等多模态终端,传输路径也从近端局域链路延伸到跨城、跨域、蜂窝接入、无线局域网和卫星通信等远距离复杂网络。TCP 作为互联网传输层的核心协议,其拥塞控制机制的性能直接影响网络资源利用效率、应用响应延迟和用户体验。

在这类端到端传输场景中,网络路径和终端侧条件呈现出更强的异构性。一方面,城域/广域网、蜂窝网络、WiFi 与卫星链路具有不同的 RTT 量级、丢包背景、可用带宽和链路抖动;另一方面,手机、电脑、边缘设备等终端在处理能力、无线接入方式、移动性和业务负载上差异显著。同一拥塞控制策略需要在短 RTT 近端高速链路和长 RTT 远距离链路之间切换,并在突发跨流量、无线误码和终端负载波动下保持稳定。上述复杂性使传统依赖固定参数或单一拥塞信号的算法难以同时满足高吞吐、低延迟和低丢包目标。

传统基于丢包的拥塞控制算法(如 NewReno [4]、CUBIC [5])将丢包作为拥塞信号,但丢包是拥塞的滞后指示器,通常意味着网络缓冲区已经溢出,排队延迟已累积到较高水平,产生“Bufferbloat”问题 [6]。基于延迟的算法(如 Vegas [7]、BBR [8])尝试通过 RTT 变化提前感知拥塞,但 RTT 测量易受操作系统调度、路径抖动等因素干扰,且与基于丢包算法共存时存在严重的公平性问题 [10]。

ECN 机制 [14] 允许网络设备在队列超过阈值时标记数据包而非丢弃,实现更早、更精确的拥塞通知。DCTCP [15] 利用 ECN 标记比例进行细粒度调整,取得了良好效果。然而,现有算法对 ECN 信号的利用仍不够充分,大多采用静态参数配置,缺乏自适应能力。

近年来,强化学习在网络拥塞控制中的应用受到关注 [17]。Aurora [19] 采用 PPO 算法训练深度神经网络策略,Orca [20] 使用强化学习动态调整 AIMD 参数。然而,现有方案存在状态空间设计不完善(通常仅 4-6 维)、未利用 ECN 和拥塞控制状态机信息、奖励函数难以平衡多目标、训练效率低等问题。

与单一网络域相比,跨地域和多接入传输的拥塞信号更难解释。丢包可能来自真实队列溢出,也可能来自无线误码或链路切换;RTT 升高既可能表示排队,也可能来自路径变化、终端调度延迟或接入链路抖动;突发 UDP/RPC/视频/备份流量还会造成短时间带宽挤占。因此,本文并不将 Swift 限定为某一类网络,而是将近

端高速、移动接入、无线局域网、广域网和卫星链路作为一组覆盖不同 RTT、带宽和终端接入条件的评估场景，考察其在多模态终端复杂性下的自适应能力。

针对上述挑战，本文提出 Swift 算法——一种基于强化学习的多信号融合自适应网络拥塞控制方法。”Swift”（雨燕）象征高吞吐量与低延迟的设计目标。本文主要贡献如下：

(1) 面向远距离与异构终端的多信号状态感知：构建 11 维有效观测子空间，将 ECN 状态、拥塞控制状态机状态、拥塞事件类型、RTT 和在途字节等协议栈信息纳入强化学习状态表示，为手机、电脑、边缘节点等多模态终端和多接入路径提供更完整的网络状态感知。

(2) 多信号融合拥塞判定与差异化响应：综合分析丢包、超时和 ECN 信号，并依据拥塞信号严重程度采用差异化窗口保留因子（丢包 0.70、ECN 0.75、超时 0.50），同时通过连续递减保护机制防止窗口过度收缩。

(3) 强化学习辅助的自适应融合控制：基于 RTT 膨胀感知阈值、奖励指数移动平均和连续增长趋势动态调整乘性增加因子 $\alpha \in [0.85, 1.30]$ ，将基于 BDP 窗口最大值滤波估计的目标窗口、拥塞窗口当前状态与管道利用率感知加权递增相结合，在吞吐、延迟和稳定性之间取得自适应折中。

2 相关工作

2.1 传统拥塞控制算法

2.1.1 基于丢包的算法

TCP Tahoe 和 Reno 是最早的拥塞控制算法 [3]，引入了慢启动、拥塞避免和快速重传/恢复机制。TCP NewReno [4] 改进了快速恢复机制，能够处理突发丢包情况，是 RFC 6582 定义的标准算法。

TCP CUBIC [5] 是 Linux 自 2.6.19 版本以来的默认算法，采用三次函数模型调整窗口：

$$W(t) = C \cdot (t - K)^3 + W_{max} \quad (1)$$

其中 $K = \sqrt[3]{W_{max} \cdot \beta / C}$ 。CUBIC 的窗口增长与 RTT 无关，在高 BDP 网络中具有更快的收敛速度。

基于丢包算法的共同缺陷包括：(1) 拥塞信号滞后，缓冲区溢出前已积累大量排队延迟；(2) 缓冲区膨胀，持续增加发送速率直至丢包；(3) 无法区分拥塞程度，对所有拥塞采用统一的窗口减半策略。

2.1.2 基于延迟的算法

TCP Vegas [7] 通过比较期望吞吐量 $cwnd/BaseRTT$ 与实际吞吐量 $cwnd/RTT$ 的差值 Δ 调整窗口，当 $\Delta < \alpha$ 时增窗， $\Delta > \beta$ 时降窗。

BBR [8] 通过估计瓶颈带宽 $BtlBw$ 和最小 RTT RTT_{prop} ，以 BDP 为目标调整发送速率： $pacing_rate = pacing_gain \times BtlBw$ 。BBR 采用周期性增益调整实现带宽探测。

Copa [9] 将目标排队延迟设为 $RTT_{standing} - RTT_{min}$ 的函数，并引入竞争模式检测机制。

这类算法面临 RTT 测量不稳定（受调度延迟、路径抖动影响）、与丢包算法共存公平性差（研究表明 BBR

与 CUBIC 共存时吞吐量可下降 30% 以上 [10])、参数敏感等问题。

2.1.3 混合算法

Compound TCP [11] 叠加基于延迟的窗口分量，Illinois [12] 根据 RTT 调整 AIMD 参数，Westwood [13] 通过 ACK 到达率估计带宽。混合算法在一定程度上缓解了单一信号的局限性，但增加了复杂性。

2.2 ECN 机制与相关算法

ECN [14] 通过 IP 报头中的 2 比特字段传递拥塞信息，定义四种码点（Non-ECT、ECT(0)、ECT(1)、CE）。发送端设置 ECT 标记表明支持 ECN；网络设备在队列超阈值时将 ECT 改为 CE；接收端回传 ECE 标志；发送端响应后设置 CWR 标志。

DCTCP [15] 利用 ECN 标记比例 α 进行细粒度调整：

$$\alpha = (1 - g) \cdot \alpha + g \cdot F, \quad cwnd = cwnd \times (1 - \alpha/2) \quad (2)$$

其中 $g = 1/16$ 是平滑因子， F 是最近一个 RTT 内被标记的数据包比例。DCTCP 能根据拥塞程度进行比例响应，但参数静态预设。HPCC [16] 利用网络设备提供的 INT 精确拥塞信息进行控制，但需要特殊硬件支持。

ECN 的优势在于：(1) 在丢包前提供明确拥塞信号；(2) 避免重传开销；(3) 信号来自网络设备显式标记，比 RTT 推测更可靠。然而，现有算法对 ECN 的利用仍不充分。

2.3 基于强化学习的拥塞控制

Aurora [19] 采用 PPO 算法训练神经网络策略，使用 10 维状态空间（延迟梯度、延迟比率、发送比率等），奖励函数为 $r = throughput - \delta \cdot delay - \lambda \cdot loss$ 。Orca [20] 在 AIMD 框架下用强化学习动态调整增加因子 α 和减少因子 β 。PCC [21] 定义实用函数 $U(x) = T \cdot x^\alpha - \delta \cdot L \cdot x - \beta \cdot D \cdot x$ ，通过梯度上升最大化。Remy [18] 采用离线优化生成规则。

现有方法的共同问题是：(1) 状态空间不完善，仅 4-6 维，未利用 ECN 和 CA 状态信息；(2) 奖励函数设计困难，多目标权衡复杂；(3) 训练效率低；(4) 泛化能力有限。Swift 通过 11 维观测空间、多信号融合和差异化响应系统性地解决这些问题。

3 Swift 算法设计

3.1 系统架构

Swift 的系统架构由四个核心模块组成。网络仿真模块基于 ns-3 [22] 构建，实现完整 TCP/IP 协议栈和多种网络拓扑（近端高速链路、哑铃型、无线/移动、广域网与卫星链路等）。拥塞控制环境模块基于 OpenGym [23] 框架实现，在 TCP 协议栈的五个关键回调点（GetSsThresh、IncreaseWindow、PktsAacked、CongestionStateSet、CwndEvent）采集 11 维状态参数，并将智能体决策应用于 TCP 连接。智能决策模块采用 Python 实现，执行多信号融合拥塞检测、差异化拥塞响应、自适应参数调优和融合控制策略计算，维护每个 TCP 连

接的独立状态。进程间通信模块采用 ZeroMQ 消息队列和 Protocol Buffers 序列化，实现仿真进程与决策进程间的同步交互；由于 OpenGym 路径依赖 ZMQ 同步通信，当前 TcpSwift 实验未启用 MTP 并行仿真。

Swift 将拥塞控制建模为马尔可夫决策过程 $\langle S, A, P, R, \gamma \rangle$ 。状态空间 S 为 11 维有效 RL 观测子空间（嵌入于 15 元素 OpenGym 传输容器，另含 4 项元数据通道），动作空间 A 直接输出新的 $ssthresh$ 和 $cwnd$ 值（连续动作空间），采用基于规则的确定性策略而非神经网络，以提高可解释性并避免神经网络推理依赖。

状态空间设计满足以下理论要求：（1）**马尔可夫性**——当前状态包含做出最优决策所需的全部信息，即 $P(s_{t+1}, r_{t+1} | s_t, a_t, s_{t-1}, \dots) = P(s_{t+1}, r_{t+1} | s_t, a_t)$ ，通过包含 ECN 状态、CA 状态等协议栈内部状态近似满足；（2）**信息充分性**——11 维空间涵盖 TCP 协议栈所有关键信息，避免因信息不足导致的次优决策；（3）**可观测性**——所有参数均可从 ns-3 标准回调接口直接获取；（4）**维度可控性**——11 维处理时间复杂度 $O(1)$ ，基于规则的决策避免了维度灾难问题。

3.2 11 维观测空间设计

Swift 的观测空间采用“11 维有效 RL 子空间 + 4 项元数据通道”的 15 元素 OpenGym 传输布局，如表1所示。参数涵盖六大类别：连接标识（socketUuid、nodeId）用于多流识别和状态隔离；时间信息（envType、simTime）用于时序分析和投递速率计算；窗口状态（ssthresh、cwnd）是拥塞控制的核心变量；传输性能（segSize、segmentsAcked、bytesInFlight）用于吞吐量估算和管道利用率计算；延迟测量（lastRtt、minRtt）用于 BDP 估算和拥塞程度判断；回调类型与拥塞控制状态（callbackType、caState、caEvent、ecnState）提供协议栈完整状态。其中前 4 项（socketUuid、envType、simTime、nodeId）为跨进程路由用的元数据，不参与窗口决策；后 11 项构成参与 AIMD/ECN 响应/ α 自适应的有效 RL 状态，故本文后续统称“11 维观测”指该有效子空间。

表 1: Swift 11 维观测空间参数定义

索引	参数	说明
0	ssthresh	慢启动阈值 (B)
1	cwnd	拥塞窗口 (B)
2	segSize	段大小 (B)
3	segmentsAcked	确认段数
4	bytesInFlight	在途字节数
5	lastRtt	最近 RTT(μs)
6	minRtt	最小 RTT(μs)
7	callbackType	回调类型 (0/1)
8	caState	CA 状态 (0-4)
9	caEvent	CA 事件 (0-5)
10	ecnState	ECN 状态 (0-5)

Swift 的核心创新在于引入三个协议栈内部状态参数。**caState** 反映 TCP 拥塞控制状态机的当前状态：CA_OPEN (0, 正常传输)、CA_DISORDER (1, 收到重复确认)、CA_CWR (2, 响应 ECN 缩减窗口)、CA_RECOVERY (3, 快速恢复中)、CA_LOSS (4, 超时丢包)。**caEvent** 表示最近的拥塞事件：包括 CA_EVENT_TX_START (0)、CWND_RESTART (1)、

COMPLETE_CWR (2)、LOSS (3)、ECN_NO_CE (4)、ECN_IS_CE (5) 六种。**ecnState** 反映 ECN 协商和标记状态：ECN_DISABLED (0)、ECN_IDLE (1)、ECN_CE_RCVD (2)、ECN_SENDING_ECE (3)、ECN_ECE_RCVD (4)、ECN_CWR_SENT (5)。

这三个参数的引入使智能体能够感知 TCP 协议栈的完整状态信息，近似满足马尔可夫性要求。所有 15 个参数均可从 ns-3 TCP 模块的标准回调接口直接获取，满足可观测性要求。11 维有效状态的处理时间复杂度为 $O(1)$ ，避免了神经网络策略的高维推理依赖。

3.3 多信号融合拥塞检测

Swift 将拥塞信号分为三类，采用两级优先级层次结构进行综合判断，并引入连续递减保护机制防止窗口过度收缩。

第一优先级——丢包与超时检测：当 $callbackType = 0$ 时（GetSsThresh 回调），表明协议栈检测到拥塞事件。此时进一步区分超时与普通丢包：若 $caState = CA_LOSS$ ，判定为超时丢包（通常意味着连续多个数据包丢失或网络路径中断）；否则判定为普通丢包（快速重传触发）。丢包和超时是最可靠的拥塞信号（似然比 $LR \approx 100$ ），一旦检测到即触发响应并重置连续递减计数器。

第二优先级——ECN 状态直接检测：当 $ecnState \in \{ECN_CE_RCVD, ECN_ECE_RCVD\}$ 时，判定存在 ECN 拥塞标记。与基于事件率统计的方法不同，Swift 直接检查协议栈的 ECN 状态字段，避免了维护时间戳队列的额外开销和事件率阈值的参数敏感性问题。

连续递减保护机制：为防止在突发拥塞期间窗口被过度收缩，Swift 维护一个连续递减计数器 d 。每次触发拥塞响应时 d 自增；每次正常窗口增长时 d 重置为零。当 d 超过最大连续递减次数 $D_{max} = 3$ 时，拥塞响应被抑制——保持当前窗口不变，避免因连续乘性递减导致窗口收缩至过小值。理论分析表明，连续 3 次丢包响应后窗口仅剩 $0.70^3 \approx 34.3\%$ ，已接近传统算法单次窗口减半的效果，进一步缩减的边际收益递减。

完整的多信号融合拥塞检测算法如算法1所示。

Algorithm 1 Multi-Signal Fusion Congestion Detection

Require: Observation obs , consecutive decrease counter d

Ensure: ($is_congested$, $type$)

```

1: // Level-1: Packet loss and timeout
2: if  $obs.callbackType = 0$  then
3:    $d \leftarrow 0$ 
4:   if  $obs.caState = CA\_LOSS$  then
5:     return ( $True$ , TIMEOUT)
6:   else
7:     return ( $True$ , LOSS)
8:   end if
9: end if
10: // Level-2: ECN state inspection
11: if  $obs.ecnState \in \{CE\_RCVD, ECE\_RCVD\}$  then
12:   return ( $True$ , ECN)
13: end if
14: return ( $False$ , NONE)

```

3.4 差异化拥塞响应机制

传统算法（如 NewReno、CUBIC）对所有拥塞信号采用统一的窗口减半策略（保留因子 $\rho = 0.5$ ），这种“一刀切”的方法无法区分轻度和严重拥塞，在轻度拥塞时响应过度（不必要地降低吞吐量），在严重拥塞时响应不足。Swift 根据拥塞类型设计差异化的窗口保留因子：

$$\begin{aligned} new_wnd &= \rho \cdot cwnd, \\ \rho &= \begin{cases} 0.70 & \text{丢包 (LOSS)} \\ 0.75 & \text{ECN 标记} \\ 0.50 & \text{超时 (TIMEOUT)} \end{cases} \end{aligned} \quad (3)$$

设计依据如下。（1）**拥塞信号的时序特性**：ECN 标记是最早的信号（队列超低阈值时触发），丢包较晚（队列溢出），超时最晚（连续丢包或路径问题）。早期信号应采用相对温和但足以排空队列的响应，晚期信号采用更激进的响应。（2）**信号可靠性**：丢包几乎不会误报（ $LR \approx 100$ ），ECN 的可靠性取决于设备配置，因此对丢包响应更坚决。（3）**吞吐量-延迟权衡**：丢包保留因子 0.70 高于传统的 0.5，配合 ECN 的提前预警机制实现更平滑的响应；当前实现将 ECN 保留因子设为 0.75，使早期拥塞预警能够触发 25% 的窗口收缩，以降低持续排队驻留风险；超时保留因子 0.50 与传统窗口减半一致，反映超时信号所指示的严重拥塞或路径故障。

慢启动阈值更新为 $new_ssthresh = \max(new_wnd, BDP)$ ，以 BDP 为下界加速恢复。窗口下限约束为 $4 \times MSS$ 。差异化响应算法如算法2所示。

Algorithm 2 Differentiated Congestion Response

Require: Current $cwnd$, congestion type $type$, consecutive decrease counter d , BDP estimate bdp
Ensure: New window new_cwnd , new threshold $new_ssthresh$

```

1:  $d \leftarrow d + 1$ 
2: if  $d > D_{max}$  then
3:    $new\_cwnd \leftarrow \max(cwnd, 4 \times MSS)$  {Freeze window}

4:  $new\_ssthresh \leftarrow new\_cwnd$ 
5: return ( $new\_cwnd$ ,  $new\_ssthresh$ )
6: end if
7: if  $type = \text{LOSS}$  then
8:    $\rho \leftarrow 0.70$ 
9: else if  $type = \text{ECN}$  then
10:   $\rho \leftarrow 0.75$ 
11: else if  $type = \text{TIMEOUT}$  then
12:   $\rho \leftarrow 0.50$ 
13: end if
14:  $new\_cwnd \leftarrow \max(\rho \times cwnd, 4 \times MSS)$ 
15:  $new\_ssthresh \leftarrow \max(new\_cwnd, bdp)$ 
16: return ( $new\_cwnd$ ,  $new\_ssthresh$ )

```

3.5 强化学习驱动的自适应参数调优

Swift 根据多种网络状态因素动态调整乘性增加因子 $\alpha \in [0.85, 1.30]$ （初始值 1.10），控制拥塞避免阶段窗

口增长速度。当前实现的调优逻辑基于 RTT 膨胀、奖励信号 EMA 和连续增长趋势三个维度。与固定阈值不同，RTT 膨胀阈值会随最小 RTT 增大而放宽，使长 RTT 路径不会因为正常的 BDP 填充过程过早触发收缩。

基于最小 RTT 感知阈值的调整：设 $r = lastRtt/minRtt$ 。Swift 首先根据 $minRtt$ 计算松弛量 $slack = 0.3 + 0.15\sqrt{\max(0, minRtt_{ms} - 1)}$ ，并构造 $low = 1 + slack$ 、 $mid = 1 + 2slack$ 、 $high = 1 + 4slack$ 三个动态阈值。当 $r < low$ 时，说明队列驻留较低， α 按 0.02 或 0.03 小步增加；当 $low \leq r < mid$ 时， α 仅增加 0.01 以维持温和探测；当 $r > high$ 时，说明排队膨胀明显， α 按 0.02 或 0.03 收缩并重置连续增长计数器。

基于奖励指数移动平均 (EMA) 的调整：为捕获较长期性能趋势，维护奖励信号的指数移动平均 $\bar{r}_t = (1 - \eta) \cdot \bar{r}_{t-1} + \eta \cdot r_t$ ，其中平滑因子 $\eta = 0.15$ 。当 $\bar{r} > 0.5$ 时， $\alpha += 0.01$ ；当 $\bar{r} < -2.0$ 时， $\alpha -= 0.01$ 。C++ 环境侧奖励由确认段数带来的吞吐收益与 RTT 膨胀惩罚共同构成，使窗口调节同时关注传输进度和排队趋势。

基于连续增长趋势的调整：窗口连续增长超过 8 次时，说明当前路径仍具备可探测空间， $\alpha += 0.01$ 。所有调整均限制在 $[0.85, 1.30]$ 内，以避免无线、蜂窝和广域网场景中的持续队列堆积。

Algorithm 3 RTT- and Reward-Aware Adaptive Parameter Tuning

Require: Current α , observation obs , consecutive increase count g , reward EMA $\bar{r}$
Ensure: Updated α

```

1:  $r \leftarrow obs.lastRtt / obs.minRtt$ 
2:  $slack \leftarrow 0.3 + 0.15\sqrt{\max(0, obs.minRtt_{ms} - 1)}$ 
3:  $low \leftarrow 1 + slack$ ;  $mid \leftarrow 1 + 2slack$ ;  $high \leftarrow 1 + 4slack$ 
4: if  $r < low$  then
5:    $\alpha \leftarrow \alpha + (obs.minRtt_{ms} > 5 ? 0.02 : 0.03)$ ;  $g \leftarrow g + 1$ 
6: else if  $r < mid$  then
7:    $\alpha \leftarrow \alpha + 0.01$ 
8: else if  $r > high$  then
9:    $\alpha \leftarrow \alpha - (obs.minRtt_{ms} > 5 ? 0.03 : 0.02)$ ;  $g \leftarrow 0$ 
10: end if
11: if  $\bar{r} > 0.5$  then
12:    $\alpha \leftarrow \alpha + 0.01$ 
13: else if  $\bar{r} < -2.0$  then
14:    $\alpha \leftarrow \alpha - 0.01$ 
15: end if
16: if  $g > 8$  then
17:    $\alpha \leftarrow \alpha + 0.01$ 
18: end if
19:  $\alpha \leftarrow \text{clamp}(\alpha, 0.85, 1.30)$ 

```

3.6 融合控制策略

Swift 将 BDP 估计、目标窗口渐进逼近和安全边界约束相结合。BDP 估计采用窗口最大值滤波器（Windowed Max-Filter）：投递速率 $DR = segmentsAcked \times segSize / lastRtt$ 存入容量为 $W_{bw} = 40$ 的滑动窗口队列，取窗口内最大值 BW_{max} 作为瓶颈带宽估计；RTT 采样窗口容量为 $W_{rtt} = 40$ ，并持续跟踪全局最小 RTT。BDP 计算为 $BDP = BW_{max} \times RTT_{min}$ ；当 BDP 估计无效时回退至当前窗口值。BDP 估计算法如算法4所

示。

Algorithm 4 Windowed Max-Filter BDP Estimation

Require: Flow state *state*, observation *obs*

Ensure: BDP estimate

```

1:  $dr \leftarrow obs.segAked \times obs.segSize / obs.lastRtt$ 
2:  $state.bw\_samples.append(dr)$  {Sliding window, capacity  $W_{bw} = 40$ }
3:  $BW_{max} \leftarrow \max(state.bw\_samples)$ 
4:  $RTT_{min} \leftarrow \min(state.min\_rtt, obs.minRtt)$ 
5:  $bdp \leftarrow BW_{max} \times RTT_{min}$ 
6: if  $bdp \leq 0$  then
7:    $bdp \leftarrow \max(obs.cwnd, 1)$  {Fallback}
8: end if
9: return  $bdp$ 

```

慢启动阶段 ($cwnd < ssthresh$)：目标窗口 $target = 2 \times BDP$ (下限 $10 \times MSS$)。Swift 采用 HyStart 风格慢启动退出：当当前 RTT 超过慢启动阶段最小 RTT 的动态阈值时，提前退出慢启动以避免首轮 BDP 填充后的突发排队；该阈值随最小 RTT 从 1.25 放宽到 1.40，使广域网和卫星链路不因正常长 RTT 特性过早退出。标准增量为 $segmentsAked \times MSS$ ；仅当当前窗口明显低于 BDP 且 RTT 尚未膨胀时，增量才提升为 $2 \times segmentsAked \times MSS$ 。

拥塞避免阶段 ($cwnd \geq ssthresh$)：当前实现先计算目标速率窗口 $target = \alpha \times BDP$ (BDP 无效时回退到当前窗口)，再以有界步长向目标靠拢，而不是直接把窗口跳变到目标值。当 $target > cwnd$ 时，窗口以 $\max(\gamma \times MSS, MSS, (target - cwnd)/16)$ 上探；当 $target < cwnd$ 时，每个 ACK 事件按超额窗口的一半逐步回落以排空队列；当二者接近时，按 $\gamma \times MSS$ 进行 Reno 式加性增长。当前实现取 $\gamma = 1.0$ 。

管道利用率感知加速与安全边界：管道利用率 $u = bytesInFlight/cwnd$ 用于识别未充分填充的链路。仅当 RTT 未明显膨胀时，Swift 才在 $u < 0.4$ 时追加 $1 \sim 2 \times MSS$ 的温和增量，并在长 RTT 路径上降低该增量，以避免无线、蜂窝和广域网场景中的队列积累。最终窗口被约束在 $[4 \times MSS, \max(4 \times BDP, 200 \times MSS)]$ 内，避免 BDP 估计陈旧时窗口失控。融合控制策略的完整算法如算法5所示。

3.7 每流独立状态管理

Swift 为每个 TCP 连接维护独立状态，通过 socketUid 为键的哈希表实现 $O(1)$ 查找。每流状态包括四个子模块：(1) **带宽与 RTT 估计模块**——容量 $W_{bw} = 40$ 的投递速率滑动窗口队列和容量 $W_{rtt} = 40$ 的 RTT 采样队列，支持窗口最大值带宽估计、全局最小 RTT 追踪和慢启动阶段最小 RTT 采样；(2) **拥塞控制计数器**——连续递减计数器 d 、连续增长计数器 g 、累计丢包和 ECN 事件计数，以及降窗后的 ACK 冻结计数；(3) **性能指标聚合**——吞吐量指数移动平均、当前 BDP 估计、上一步窗口和时间戳；(4) **自适应参数**—— α (初始 1.10)、 γ (默认 1.0) 和慢启动标志位。

状态管理遵循延迟初始化 (首次访问时创建) 和增量更新 (仅更新变化字段) 两个原则。每流独立管理避免了多流并发场景下的流间干扰，支持差异化控制。全局共享奖励 EMA (平滑因子 $\eta = 0.15$) 捕获跨流的整

Algorithm 5 Hybrid Rate-Window Fusion Control

Require: Observation *obs*, parameter α , BDP estimate bdp

Ensure: New window new_cwnd

```

1: if  $obs.cwnd < obs.ssthresh$  and flow is in slow start then
2:   Apply HyStart-style RTT-inflation exit
3:    $target \leftarrow \max(2 \times bdp, 10 \times MSS)$ 
4:    $\Delta \leftarrow obs.segmentsAked \times MSS$ 
5:   if  $obs.cwnd < 0.3 \times bdp$  and RTT is not inflated then
6:      $\Delta \leftarrow 2 \times obs.segmentsAked \times MSS$ 
7:   end if
8:    $new\_cwnd \leftarrow \min(obs.cwnd + \Delta, target)$ 
9: else
10:   $target \leftarrow \alpha \times bdp$ 
11:  if  $target > obs.cwnd$  then
12:     $new\_cwnd \leftarrow obs.cwnd + \max(MSS, (target - obs.cwnd)/16)$ 
13:  else if  $target < obs.cwnd$  then
14:     $new\_cwnd \leftarrow obs.cwnd - \max((obs.cwnd - target)/2, MSS)$ 
15:  else
16:     $new\_cwnd \leftarrow obs.cwnd + MSS$ 
17:  end if
18: end if
19: Apply utilization-aware boost only when RTT is not inflated
20:  $new\_cwnd \leftarrow \text{clamp}(new\_cwnd, 4 \times MSS, \max(4 \times bdp, 200 \times MSS))$ 
21: return  $new\_cwnd$ 

```

体性能趋势。

4 实验评估

4.1 实验配置

硬件与软件：Intel i7-10700K 8 核 CPU, 32GB DDR4 内存, Ubuntu 20.04, ns-3.40 [22], OpenGym [23], Python 3.8。

网络拓扑：经典哑铃型拓扑——3 个发送端经接入链路连接路由器 R0, R0 通过瓶颈链路 (DropTail/RED 队列) 连接 R1, R1 经接入链路连接 3 个接收端。使用 RED 队列，标记阈值为队列长度的 30%。

仿真参数：仿真时长 20 秒, MSS=1460 字节, 初始窗口 10 个 MSS (RFC 6928), 启用 ECN 和 SACK, 随机种子 42, 每配置重复 10 次取平均。

实验矩阵：为全面评估 Swift 在不同背景流量条件下的鲁棒性，仿真器设计了两类相互独立但严格对照的实验设置：

- **纯 TCP 设置 (无 UDP Burst)** : $enable_udp_burst=false$, 瓶颈链路上仅承载 3 条 TCP 长流, 对应数据文件 logs/plots/summary.csv.
- **UDP Burst 干扰设置**: $enable_udp_burst=true$, 在同一瓶颈上并发启动一路 800 Mbps、50% 占空比 (On 0.5 s / Off 0.5 s) 的 OnOff UDP 突发源 (1024 字节报文), 模拟移动终端后台同步、视频传输、RPC 突发、备份任务或多租户共享链路中的典型跨流量干扰, 对应数据文件 logs/plots-udp/summary.csv.

两组实验共享完全相同的 21 场景配置、哑铃拓扑、随机种子与评测指标, 仅 $enable_udp_burst$ 参数取

值不同，由此构成严格的 A/B 对照矩阵，用于分别评估稳态性能（第 4.3 节）与跨流量鲁棒性（第 4.5 节）。

对比算法：NewReno [4]（经典 AIMD）、CUBIC [5]（Linux 默认）、BBR [8]（Google 基于延迟）。

评测指标：吞吐量 $T = TotalBytes/SimTime$ (Mbps)、平均延迟 (ms)、Jain 公平性指数 $J = (\sum x_i)^2 / (n \cdot \sum x_i^2)$ [25]、丢包率。

4.2 实验场景设计

为全面评估 Swift 在不同网络环境下的性能，设计了覆盖近端高速链路、无线接入、移动网络、广域网和卫星通信的 21 种场景。其中 9 种场景具备全部四种协议的完整对比数据，如表 2 所示。哑铃型拓扑（3 对发送/接收节点，DropTail 队列）的接入带宽和瓶颈带宽根据场景独立配置。每组实验使用 10 个随机种子重复运行，取平均值。吞吐量报告为 3 条流的聚合值以反映瓶颈链路利用情况。

表 2: 9 种主要实验场景配置

编号	场景名称	类型	瓶颈/RTT
S1	intra_rack_10g	近端高速	10G, μ s
S2	intra_rack_25g	近端高速	25G, μ s
S3	leaf_spine_20g	近端高速	20G, μ s
S4	nr_5g_emb	5G eMBB	500M, ms
S5	wan_metro	城域 WAN	1G, ms
S6	wifi_ac	802.11ac	400M, ms
S7	wifi_ax	802.11ax	600M, ms
S8	satellite_geo	GEO 卫星	低带宽, 300+ms
S9	nr_5g_edge	5G 边缘	100M, 20+ms

4.3 主要实验结果

本小节报告纯 TCP 设置（无 UDP Burst）下的稳态性能。跨流量鲁棒性见第 4.5 节。

表 3 和表 4 分别汇总了 9 个场景的吞吐量和延迟对比数据。所有数值从 ns-3 FlowMonitor 导出，吞吐量为 3 条并发流的聚合值。

表 3: 聚合吞吐量对比 (Mbps) 及 Swift 相对 NewReno/CUBIC 的差距

场景	Swift	NR	CUBIC	BBR	差距
S1	10213	10214	10213	2.3	-0.01%
S2	25532	25533	25533	2.2	-0.00%
S3	20409	20410	20409	7544	-0.00%
S4	506.3	507.4	508.4	509.5	-0.3%
S5	1018	1020	1020	1023	-0.2%
S6	405.6	406.0	406.8	406.7	-0.3%
S7	608.8	610.1	610.9	612.1	-0.2%
S8	8.71	5.73	7.80	2.90	+28.8%
S9	101.4	101.2	101.5	100.8	+0.1%

吞吐量分析：Swift 在 S1~S7 场景的吞吐量与 NewReno/CUBIC 差距均在 0.4% 以内，管道利用率接近饱和。在 GEO 卫星场景 (S8) 中，Swift 聚合吞吐量为 8.71 Mbps，相对 NewReno (5.73 Mbps)、CUBIC (7.80 Mbps) 与 BBR (2.90 Mbps) 分别提升 52.0%、11.7% 与 200.3%，相对 NewReno/CUBIC 均值提升 28.8%。其零丢包特性避免了高延迟链路上的重传开销，是取得该优势的根本原因。BBR 则在近端高速场景 (S1~S3) 出现显著的吞吐量退化，10G 与 25G 微秒级 RTT 场景下

表 4: 平均端到端延迟对比 (ms)

场景	Swift	NR	CUBIC	BBR	vs NR/C
S1	0.356	1.974	2.258	0.005	-83.2%
S2	0.142	1.898	2.197	0.005	-93.1%
S3	0.178	1.907	2.224	0.010	-91.4%
S4	7.17	9.24	9.72	12.19	-24.4%
S5	3.57	4.61	4.69	3.73	-23.2%
S6	8.93	9.55	10.01	11.56	-8.7%
S7	5.97	7.28	7.59	8.64	-19.7%
S8	355.7	342.6	370.1	466.1	-0.2%
S9	29.35	20.32	21.61	39.21	+40.0%

吞吐量跌至 ~ 2 Mbps，这与文献 [10] 的报告一致——在极短 RTT 环境中，BBR 的 ProbeRTT/ProbeBW 周期性探测产生严重测量噪声，从而导致多流竞争下的带宽估计崩塌。

延迟分析：Swift 的延迟优势呈现明显的场景依赖性。在近端高速场景 (S1~S3)，排队延迟占端到端延迟的主要部分，Swift 通过 ECN-aware 的提前窗口调整将延迟从 1.9~2.3 ms 降至 0.14~0.36 ms，降幅 83%~93%。在 5G 和城域 WAN 场景 (S4、S5)，传播延迟占比增大，Swift 仍实现 23%~24% 的延迟降低。在无线场景 (S6、S7)，延迟降低 9%~20%。在 5G 边缘场景 (S9)，Swift 延迟相较 NewReno 存在约 40% 的小幅度抬升——其机理在于 Swift 的 $\alpha \times BDP$ 目标窗口在 20+ ms RTT 环境下的扩张幅度仍有进一步收紧空间，可能在瓶颈链路引入轻微的队列驻留。第 5.3 节将进一步讨论该适用边界。

丢包率分析：Swift 在全部 9 个场景均实现零丢包 (0.00%)，而 NewReno 在 S8 中丢包 0.38%，CUBIC 在 S8 中丢包 0.76%，BBR 在 S8 中丢包 5.88%。表 5 展示了丢包率对比。零丢包特性是 Swift 差异化拥塞响应机制的直接结果：ECN 保留因子 $\rho_{ecn} = 0.75$ 在队列溢出前即缓解拥塞，连续递减保护与降窗后冻结机制共同抑制窗口过度收缩后的快速反弹。

表 5: 丢包率对比 (%)

场景	Swift	NR	CUBIC	BBR
S1~S7	0.00	≤ 0.01	≤ 0.02	≤ 0.09
S8 (GEO)	0.00	0.38	0.76	5.88
S9 (5G 边缘)	0.00	0.01	0.02	0.05

4.4 深入分析

4.4.1 场景分类讨论

将 9 个场景按 Swift 的性能表现分为三类。

A 类——近端高速链路 (S1~S3)：瓶颈带宽 ≥ 10 Gbps，端到端 RTT 在微秒量级。排队延迟占端到端延迟的 90% 以上，Swift 的 ECN-aware 窗口调整直接作用于主要延迟成分。以 S2 (25G 微秒级 RTT) 为例，NewReno/CUBIC 平均延迟 2.05 ms，其中排队分量约 1.9 ms；Swift 通过在 ECN 标记时仅收缩 15% 窗口将队列维持在极低水位，排队延迟降至 0.07 ms (降幅 96%)，端到端延迟 0.14 ms (降幅 93%)。BBR 在该类场景出现严重吞吐量退化 (~ 2 Mbps)，原因是微秒级 RTT 导致其 ProbeRTT/ProbeBW 周期性探测产生大幅测量噪

声——同类退化已在 Hock 等 [10] 的实测中得到印证。

B 类——中速网络 (S4~S7)：瓶颈带宽 100 M~1 Gbps, RTT 在毫秒量级。传播延迟占比升高, 排队延迟优化空间相应缩小。Swift 延迟降低幅度从 24% (S4, 5G eMBB) 到 9% (S6, WiFi AC) 不等。该类场景 Swift 的核心价值在于零丢包特性：差异化 ECN 响应在队列溢出前完成拥塞缓解, 而 NewReno/CUBIC 依赖丢包触发的乘性递减在高 BDP-delay 积环境中反应滞后。

C 类——高延迟/低带宽 (S8~S9)：包括 GEO 卫星 (RTT≈600 ms) 和 5G 边缘 (RTT≈40 ms)。S8 中 Swift 凭借零丢包避免了高延迟链路上的重传惩罚, 聚合吞吐量相对 NewReno/CUBIC 均值提升 28.8% (相对 NewReno 单算法 52.0%、相对 BBR 200.3%)。S9 则呈现 Swift 的 $\alpha \times BDP$ 窗口目标在中等 RTT 环境下的适用边界——当 $\alpha > 1.0$ 时目标窗口略超过 BDP, 可能令缓冲区在稳态下出现轻度驻留。

4.4.2 机制一致性与证据边界

差异化响应机制：当前实现将拥塞信号分为丢包、ECN 和超时三类, 并分别采用 0.70、0.75 和 0.50 的窗口保留因子。该设计与日志中的零丢包现象一致：ECN 在队列溢出前触发温和但更及时的窗口收缩, 丢包和超时则采用更强响应以快速排空队列。由于当前仓库未保留可复现实验矩阵中的独立消融重跑结果, 本文不把差异化响应相对统一保留因子的收益写作独立定量化结论。

连续递减保护与冻结机制：当前实现设置 $D_{max} = 3$, 并在降窗后冻结 4 个 ACK 事件的窗口增长, 使连续拥塞信号不会在短时间内反复触发无界收缩, 也不会降窗后立即反弹形成队列振荡。以连续 ECN 事件为例, 三次响应后的窗口保留比例为 $0.75^3 \approx 0.42$, 随后保护机制进入保持行为；这一定性机制解释了 Burst 场景下 Swift 维持零丢包的趋势, 但本文不声明未重跑消融实验所得的百分比收益。

α 自适应机制：当前实现采用 $\alpha \in [0.85, 1.30]$ 、初始值 1.10, 并使用最小 RTT 感知阈值、奖励 EMA 和连续增长趋势共同调节窗口增长偏置。与固定三级阈值相比, 最小 RTT 感知阈值可在 WAN 和卫星链路上放宽 RTT 膨胀容忍度, 同时可在无线、蜂窝等更易排队的场景下保留较强的排队抑制能力。由于日志目录中没有保存固定 α 、固定 RTT 阈值或不同 α 边界的完整对照结果, 本文仅将其作为实现机制和适用边界分析, 不给出无来源的参数扫描数字。

4.5 UDP Burst 跨流量鲁棒性评估

多模态终端与多租户共享链路普遍承载突发性非响应式 UDP 流量 (视频、RPC、后台同步、备份、游戏控制等)。为评估 Swift 在这类跨流量干扰下的鲁棒性, 本节在严格对照的 A/B 实验矩阵下对比四种协议的性能变化。UDP Burst 配置为 800 Mbps、50% 占空比 (On 0.5 s / Off 0.5 s) 的 1024 字节 OnOff 源, 与 TCP 长流共享同一瓶颈队列 (logs/plots-udp/summary.csv)。

4.5.1 聚合鲁棒性指标

在 17 个在两种设置下均具备四协议完整数据的场景上聚合, 得到表 6 所示的跨流量鲁棒性指标。

表 6: UDP Burst 跨流量鲁棒性聚合指标 ($n = 17$)

协议	均 ΔT	中 ΔT	均 ΔD	最大 ΔL	零丢包
TcpSwift	19.78%	23.75%	+12.32%	0.000%	17/17
TcpNewReno	24.37%	23.75%	+3.27%	+0.300%	7/17
TcpCubic	21.36%	23.74%	+2.39%	+3.450%	9/17
TcpBbr	55.55%	64.06%	+39.95%	+1.580%	10/17

ΔT 为 UDP Burst 引入后吞吐量下降百分比; ΔD 为平均延迟增幅; ΔL 为新丢包率; 零丢包率统计 $Loss < 0.01\%$ 的场景数。

三项核心观察：第一, **Swift 是唯一在全部 17 个场景中维持零丢包的算法**, CUBIC 在 lte_poor 下丢包率从 1.12% 恶化至 4.57% ($\Delta L = +3.45\%$), BBR 在 satellite_geo 下从 5.88% 恶化至 7.46% ($\Delta L = +1.58\%$); 第二, **Swift 的吞吐量下降幅度最小 (19.78%)**, BBR 因带宽探测信号在跨流量下严重失真而出现近 55% 的崩塌式退化; 第三, **Swift 延迟存在小幅度抬升 (约 12.32%)**, 相较 NewReno/CUBIC 的 $\sim 3\%$ 略高, 其机理在于 Swift 基于 $\alpha \times BDP$ 的窗口目标在 UDP 抖动放大的 RTT 噪声下倾向于令缓冲区出现轻度驻留——相关分析见第 5.3 节。

4.5.2 头部场景逐一验证

表 7 给出 9 个主要场景中在两种实验设置下均具备完整四协议数据的 6 个场景的对照结果。

关键发现：

- **近端高速链路场景的延迟优势被 UDP Burst 完全保留：**S1~S3 中 Swift 相对 NewReno/CUBIC 均值的延迟降幅在带 Burst 条件下仍维持于 -82.28% (S1)、-92.91% (S2)、-91.02% (S3), 与无 Burst 条件的 -83.2%/-93.1%/-91.4% 基本一致。其根本原因在于：微秒级 RTT 下 UDP Burst 引入的排队抖动远小于固定偏移的传统算法在 PfifoFast 队列中造成的稳态排队, Swift 基于 ECN 的快速响应仍有效。
- **GEO 卫星场景的吞吐量优势被 UDP Burst 显著放大：**S8 中无 Burst 时 Swift 相对 NewReno/CUBIC 均值提升 +28.8%, 加入 Burst 后提升至 +86.8% (5.80 Mbps vs 3.10 Mbps)。机理为：高 RTT 链路上单次丢包的重传代价达一个完整 RTT (≈ 600 ms), Burst 使 NewReno/CUBIC 的丢包率-损失率乘积显著扩大, 而 Swift 的零丢包特性使其免疫于此损失。
- **BBR 在跨流量下出现系统性崩塌：**在 1 Gbps 级对称链路 (asymmetric_high、congested_heavy、symmetric_low 等), BBR 吞吐量从 ~ 520 Mbps 骤降至 ~ 57 Mbps (降幅 $\sim 89\%$), wifi_n 场景下从 49.87 Mbps 降至 4.94 Mbps (降幅 $\sim 90\%$), 印证了跨流量下 BBR 带宽探测的已知脆弱性 [10]。
- **Swift 延迟在 S5/S9 出现温和膨胀：**wan_metro 延迟从 3.57 ms 升至 4.45 ms (+24.86%), nr_5g_edge 从 29.35 ms 升至 32.89 ms (+12.06%)。机理分析见第 5.3 节。

4.5.3 低速/无线接入场景的零丢包补充验证

除上述 6 个头部场景外, UDP Burst A/B 矩阵在低速蜂窝与传统 WiFi 信道下还呈现三组与基线形成显著差距的零丢包对照数据 (数据均直接溯源至 logs/plots-udp/summary.csv)：

- **lte_poor：**Swift 7.48 Mbps、丢包率 0.00%; CUBIC 同条件下从 1.12% 恶化至 4.57% ($\Delta L = +3.45\%$, 全

表 7: 9 个主要场景在 UDP Burst 对照下的性能 (无干扰 / 有干扰)

场景	协议	吞吐量 (Mbps)		延迟 (ms)		丢包率 (%)	
		无	有	无	有	无	有
S1 intra_rack_10g	Swift	10212.94	9811.36	0.356	0.370	0.00	0.00
	NewReno	10213.82	9812.27	1.974	1.954	0.01	0.01
	CUBIC	10213.42	9811.88	2.258	2.225	0.00	0.00
	BBR	2.28	2.27	0.005	0.005	0.00	0.00
S2 intra_rack_25g	Swift	25532.34	25131.00	0.142	0.145	0.00	0.00
	NewReno	25533.31	25131.96	1.898	1.887	0.00	0.00
	CUBIC	25533.13	25131.79	2.197	2.192	0.00	0.00
	BBR	2.24	2.24	0.005	0.005	0.00	0.00
S3 leaf_spine_20g	Swift	20408.76	20007.04	0.178	0.182	0.00	0.00
	NewReno	20409.65	20007.96	1.907	1.891	0.00	0.00
	CUBIC	20409.49	20007.77	2.224	2.154	0.00	0.00
	BBR	7543.93	7555.38	0.010	0.010	0.00	0.00
S5 wan_metro	Swift	1017.73	815.02	3.567	4.454	0.00	0.00
	NewReno	1019.57	816.22	4.610	4.572	0.00	0.00
	CUBIC	1019.89	817.05	4.689	4.461	0.00	0.00
	BBR	1022.54	736.28	3.729	3.791	0.00	0.00
S8 satellite_geo	Swift	8.71	5.80	355.74	425.38	0.00	0.00
	NewReno	5.73	2.03	342.61	373.59	0.38	0.11
	CUBIC	7.80	4.18	370.14	378.81	0.76	0.24
	BBR	2.90	1.86	466.08	430.75	5.88	7.46
S9 nr_5g_edge	Swift	101.37	77.03	29.35	32.89	0.00	0.00
	NewReno	101.16	70.70	20.32	20.94	0.01	0.02
	CUBIC	101.45	75.57	21.61	21.26	0.02	0.03
	BBR	100.78	44.42	39.21	31.76	0.05	0.19

部 17 场景中最大新增丢包)；NewReno 6.48 Mbps、丢包 0.24%；BBR 6.38 Mbps、丢包 2.49%。

- **lte_good**: Swift 37.86 Mbps、丢包率 0.00%，是该场景下唯一保持零丢包的算法；CUBIC 37.28 Mbps、丢包率从 0.66% 恶化至 1.79%；BBR 17.13 Mbps、丢包率 0.57% (吞吐量为 Swift 的 45%)。
- **wifi_n**: Swift 38.54 Mbps、丢包率 0.00%；BBR 在 Burst 下从 49.87 Mbps 崩塌至 4.94 Mbps ($\Delta T \approx -90\%$)，丢包率从 0.09% 升至 1.22%；CUBIC 37.79 Mbps、丢包 0.06%。

上述三场景与表 7 的 6 个头部场景互不重叠，进一步证明 Swift 的零丢包鲁棒性在低速蜂窝 (~10 Mbps) 与传统 WiFi (~50 Mbps) 信道、以及在丢包率本就较高的存量基线之上同样成立。其机制根因仍为差异化 ECN 响应 ($\rho_{ecn} = 0.75$) 的提前预警与 BDP 窗口化最大值滤波对 Burst 抖动的低通滤波，详见下节鲁棒性机理分析。

4.5.4 鲁棒性机理分析

Swift 在 UDP Burst 下的鲁棒性源于三项协同机制：第一，**ECN 温和响应** ($\rho_{ecn} = 0.75$) 在 UDP Burst 触发的早期拥塞预警下避免了 NewReno/CUBIC 式的乘性减半，维持了稳定吞吐量；第二，**差异化丢包响应** ($\rho_{loss} = 0.70$ 、 $\rho_{timeout} = 0.50$) 在 Burst 引发真正丢包时按严重程度精细收缩，避免了过度退避；第三，**BDP 窗口化最大值滤波**对 Burst 引入的瞬时带宽抖动具有低通滤波特性，使 BDP 估计保持于 Burst 间歇期的真实瓶颈能力附近，避免了 BBR 式的带宽探测崩塌。零丢包特性则是上述机制的综合结果——ECN 在队列溢出前完成窗口收敛。

5 讨论

5.1 理论分析

收敛性: Swift 的窗口动态可用 Lyapunov 函数 $V = \frac{1}{2}(cwnd - cwnd^*)^2$ 进行定性分析，其中 $cwnd^* \approx BDP$ 。在拥塞避免阶段，窗口不再直接跳变到 $\alpha \times BDP$ ，而是以有界步长向目标窗口渐进逼近；当检测到 ECN、丢包或超时，窗口分别按 0.75、0.70 和 0.50 的保留因子收缩。连续递减保护 ($D_{max} = 3$) 与降窗后 4 个 ACK 事件的冻结机制进一步抑制重复收缩与快速反弹。 α 的边界约束 $[0.85, 1.30]$ 和小步长调整保证参数不会在单个 RTT 内发生突变。

复杂度与计算开销: 当前实现采用基于规则的确定性策略，单次决策仅包含固定数量的状态读取、分支判断、窗口最大值更新和哈希表查询，时间复杂度为 $O(1)$ ；每流状态主要由带宽样本队列、RTT 样本队列、拥塞计数器、窗口历史和少量自适应参数构成，空间复杂度为 $O(n \cdot (W_{bw} + W_{rtt}))$ ，其中 n 为并发流数， $W_{bw} = 40$ 、 $W_{rtt} = 40$ 。由于当前仓库未保存独立的推理延迟、CPU 利用率和内存压测日志，本文仅给出复杂度级别分析，不声称具体硬件平台上的毫秒级延迟或大规模并发资源占用数字。

信息论视角: 多信号融合的两级优先级等价于按似然比排序。设网络拥塞状态 $C \in \{0, 1\}$ ，各信号条件独立，则联合似然比 $LR(\mathbf{S}) = \prod_i P(S_i | C = 1) / P(S_i | C = 0)$ 。丢包/超时 $LR \approx 100$ ，ECN $LR \approx 20$ 。优先使用高似然比信号可最小化贝叶斯误检概率 P_e 。连续递减保护机制从信息论角度可理解为对连续同类信号的置信度递减——第 k 次连续信号的增量信息量 $I_k \propto 1/k$ ，当 $k > D_{max}$ 时信息增益不足以证明进一步缩减的合

理性。

最优控制视角：差异化保留因子可从最优控制理论分析。严重拥塞（超时，连续丢包或路径故障）需最大窗口缩减（ $\rho = 0.50$ ）快速清空队列；中度拥塞（丢包，队列溢出）采用中等缩减（ $\rho = 0.70$ ）；轻度拥塞（ECN，队列开始积累）采用相对温和但足以抑制队列扩张的缩减（ $\rho = 0.75$ ）。Swift 的三类差异化设计近似了不同拥塞程度下的最优控制输入，响应强度与拥塞严重程度正相关。

5.2 与相关工作的详细对比

表 8: Swift 与主要相关算法的特性对比

特性	Swift	Aurora	DCTCP
状态空间维度	15	10	N/A
ECN 状态利用	是	否	是
CA 状态利用	是	否	否
决策机制	规则	DNN	公式
可解释性	高	低	高
训练需求	无	小时/天	无
在线计算形式	规则分支与算术	神经网络推理	ECN 比例公式
差异化响应	是	否	否
参数自适应	是	固定	固定

vs. Aurora [19]: Swift 采用 11 维有效状态子空间 (vs. Aurora 常用 10 维观测)，并引入 ECN 和 CA 状态参数。Swift 采用基于规则的确定性决策 (vs. 深度神经网络)，无需训练过程即可部署，且决策路径更易解释和调试。Aurora 的优势在于神经网络的强表示能力，但在实际部署中面临训练数据收集、模型更新和推理开销等挑战；Swift 则选择以可解释规则和协议栈内部信号换取更明确的工程可控性。

vs. DCTCP [15]: Swift 直接检测 ECN 协议栈状态进行离散拥塞判断 (vs. 标记比例 α 连续调整)，实现差异化响应（丢包 0.70/ECN 0.75/超时 0.50 vs. 统一公式 $1 - \alpha/2$ ），综合利用丢包、ECN 和 CA 状态三类信号 (vs. 仅 ECN)，并实现参数自适应 (vs. 静态配置)。连续递减保护机制进一步增强了在突发拥塞下的鲁棒性。

vs. BBR [8]: Swift 综合多种显式拥塞信号 (vs. 仅 RTT 测量)，在近端高速场景 (S1~S3) 维持 >99.9% 带宽利用率 (vs. BBR 退化至 <0.1%)，在中速场景吞吐量差距 <0.4%。BBR 在微秒级 RTT 的多流竞争中因 ProbeRTT/ProbeBW 周期性探测的测量噪声导致严重退化，实验中在 S1 (10G 微秒级 RTT) 和 S2 (25G 微秒级 RTT) 均降至 ~2 Mbps。

5.3 适用边界与未来方向

中低速与中等 RTT 延迟抬升：实验显示 Swift 在中低速网络与中等 RTT 场景中存在小幅度延迟抬升。在 dc_100m 场景延迟相较 NewReno/CUBIC 存在约 123% 的小幅抬升，wifi_legacy 场景约 51%，5G 边缘场景约 40%。其机理在于 $\alpha \times BDP$ 目标窗口机制：当 $\alpha > 1.0$ 时目标窗口略超过 BDP，ECN 的 $\beta = 0.75$ 保留因子仍可能在部分中低速链路上留下轻度队列驻留空间。可进一步探索的优化方向包括：(1) 引入排队延迟显式估计 ($q_{est} = RTT_{smoothed} - RTT_{min}$) 作为 α 调整的额外输入；(2) 在 RTT 膨胀持续超过最小 RTT 感知阈值

时自动将 α 收敛至 1.0 以下；(3) 采用多时间尺度 BDP 估计区分传播延迟与排队延迟。

广域网吞吐量进一步上探空间：cross_dc_wan 场景中 Swift 吞吐量相对 NewReno/CUBIC 存在约 29% 的小幅回落。分析表明高 RTT 环境 (5 ms) 下 BDP 估计的瞬时值偏保守，当前 α 自适应范围 [0.85, 1.30] 和目标窗口渐进逼近策略仍有进一步场景化调优空间。后续可考虑为 WAN 场景启用更积极的带宽探测策略或更细粒度的 α 更新规则，但相关调整需通过完整实验矩阵重跑后才能形成新的定量结论。

参数预设依赖：保留因子 (0.70/0.75/0.50)、 α 边界 ([0.85, 1.30])、连续递减阈值 ($D_{max} = 3$) 和降窗后冻结窗口增长的 ACK 事件数 (4) 为当前实现中的手动调优预设值。未来可通过元学习 [26] 或贝叶斯优化实现参数的在线自动搜索。

深度 RL 增强：当前基于规则的策略可解释性强但表示能力有限。可探索离线强化学习训练神经网络策略，再通过模型蒸馏提取为规则，兼顾表示能力和实时性。

真实部署验证：需将算法实现为 Linux 内核 TCP 模块或 QUIC 用户态协议栈，在覆盖近端高速、蜂窝/无线接入、广域网与卫星链路的真实或半实物测试床上验证。仿真与真实环境的差异 (如 NIC offload、中断合并、NUMA 效应、终端操作系统调度和无线链路波动) 可能影响实际性能。

证据边界说明：本文所有定量结论均基于经数据质量审计后的 logs/plots/summary.csv 与 logs/plots-udp/summary.csv。上述适用边界用于界定当前实验结论的适用范围：中低速与中等 RTT 场景中的延迟抬升、以及部分广域网场景中的吞吐回落不被纳入头部优势结论，而仅作为算法机制分析和参数敏感性讨论的依据。任何后续参数或控制策略调整，只有在完成同一实验矩阵重跑、异常筛查和结果溯源后，才能作为新的定量结论引用。

6 结论

本文提出了 Swift——一种基于强化学习辅助的多信号融合自适应网络拥塞控制算法，面向远距离传输和手机、电脑、边缘设备等多模态终端并存的复杂端到端网络。Swift 的核心技术贡献收敛为三点：(1) 面向远距离与异构终端的 11 维有效观测空间设计，将 ECN 状态、CA 状态机、拥塞事件、RTT 和在途字节等协议栈内部状态纳入强化学习状态表示；(2) 多信号融合拥塞判定与差异化响应机制，综合丢包、ECN 和超时信号，并配合连续递减保护 ($D_{max} = 3$) 与降窗后冻结机制抑制窗口过度收缩和快速反弹；(3) 强化学习辅助的自适应融合控制策略，结合最小 RTT 感知阈值、奖励信号 EMA、BDP 窗口化最大值滤波估计、目标窗口渐进逼近与当前窗口保守控制，在吞吐、延迟和稳定性之间动态折中。

在 ns-3 平台覆盖近端高速、无线接入、5G 移动通信、WAN 与卫星通信的 21 个场景、84 组实验中，Swift 在近端高速场景 (S1~S3) 保持与 NewReno/CUBIC 等价的吞吐量 (差距 <0.01%) 同时将排队延迟降低 83%~93%；

在中速网络场景 (S4~S7) 延迟降低 9%~24% 且全部实现零丢包; 在 GEO 卫星长距离场景 (S8) 凭借零丢包特性相对 NewReno/CUBIC 均值实现吞吐量提升 28.8% (相对 NewReno 单算法为 52.0%, 相对 BBR 为 200.3%)。进一步地, 在 800 Mbps UDP Burst 的 A/B 对照跨流量鲁棒性实验中, Swift 在全部 17 个完整对照场景中维持零丢包 (CUBIC 最大新增丢包率 +3.45%、BBR +1.58%), 吞吐量回落幅度最小 (均值仅 19.78%, BBR 为崩塌式 55.55%), 并且近端高速场景的延迟降幅与 GEO 卫星场景的吞吐量优势在跨流量条件下不但被完整保留, 后者更从 +28.8% 进一步扩大至 +86.8%。

实验同时如实披露了 Swift 在中低速与中等 RTT 场景下存在小幅度延迟抬升、以及部分跨域广域网场景下吞吐量存在小幅回落等适用边界, 其机理在于 $\alpha \times BDP$ 目标窗口机制在高 RTT 环境下的扩张幅度仍有进一步收紧空间, 以及 BDP 估计在长 RTT 路径上的瞬时值偏保守。这些发现为后续引入排队延迟感知的 α 动态调整、多时间尺度 BDP 估计以及更积极的带宽探测策略提供了明确的进一步优化方向。总体而言, Swift 以可解释的规则化控制取得了接近深度强化学习方案的性能表现, 为远距离传输、多接入链路与多模态终端并存环境下的拥塞控制提供了一种高效、自适应、可解释且具备显著跨流量鲁棒性的解决方案。

参考文献

- [1] IDC. Worldwide Global DataSphere Forecast, 2023-2027. IDC Report, 2023.
- [2] Benson T, Akella A, Maltz D A. Network traffic characteristics of data centers in the wild. In Proc. IMC, 2010: 267-280.
- [3] Jacobson V. Congestion avoidance and control. ACM SIGCOMM CCR, 1988, 18(4): 314-329.
- [4] Floyd S, Henderson T, Gurtov A. The NewReno modification to TCP's fast recovery algorithm. RFC 6582, 2012.
- [5] Ha S, Rhee I, Xu L. CUBIC: a new TCP-friendly high-speed TCP variant. ACM SIGOPS OSR, 2008, 42(5): 64-74.
- [6] Gettys J, Nichols K. Bufferbloat: Dark buffers in the Internet. Commun. ACM, 2012, 55(1): 57-65.
- [7] Brakmo L S, Peterson L L. TCP Vegas: End to end congestion avoidance on a global Internet. IEEE JSAC, 1995, 13(8): 1465-1480.
- [8] Cardwell N, Cheng Y, et al. BBR: Congestion-based congestion control. ACM Queue, 2016, 14(5): 20-53.
- [9] Arun V, Balakrishnan H. Copa: Practical delay-based congestion control for the Internet. In Proc. NSDI, 2018: 329-342.
- [10] Hock M, Bless R, Zitterbart M. Experimental evaluation of BBR congestion control. In Proc. ICNP, 2017: 1-10.
- [11] Tan K, Song J, et al. A compound TCP approach for high-speed and long distance networks. In Proc. INFOCOM, 2006: 1-12.
- [12] Liu S, Başar T, Srikant R. TCP-Illinois: A loss-and delay-based congestion control algorithm. Perf. Eval., 2008, 65(6-7): 417-440.
- [13] Mascolo S, Casetti C, et al. TCP Westwood: Bandwidth estimation for enhanced transport over wireless links. In Proc. MobiCom, 2001: 287-297.
- [14] Ramakrishnan K, Floyd S, Black D. The addition of ECN to IP. RFC 3168, 2001.
- [15] Alizadeh M, Greenberg A, et al. Data center TCP (DCTCP). ACM SIGCOMM CCR, 2010, 40(4): 63-74.
- [16] Li Y, Miao R, et al. HPCC: High precision congestion control. In Proc. SIGCOMM, 2019: 44-58.
- [17] Jay N, Rotman N, et al. A deep reinforcement learning perspective on Internet congestion control. In Proc. ICML, 2019: 3050-3059.
- [18] Winstein K, Balakrishnan H. TCP ex machina: Computer-generated congestion control. ACM SIGCOMM CCR, 2013, 43(4): 123-134.
- [19] Jay N, Rotman N, et al. Internet congestion control via deep reinforcement learning. In Proc. NeurIPS, 2019.
- [20] Abbasloo S, Yen C Y, Chao H J. Classic meets modern: A pragmatic learning-based congestion control. In Proc. SIGCOMM, 2020: 632-647.
- [21] Dong M, Li Q, et al. PCC: Re-architecting congestion control for consistent high performance. In Proc. NSDI, 2015: 395-408.
- [22] Riley G F, Henderson T R. The ns-3 network simulator. Springer, 2010: 15-34.
- [23] Gawłowicz P, Zubow A. ns-3 meets OpenAI Gym: The playground for ML in networking research. In Proc. MSWiM, 2019: 113-120.
- [24] Afanasyev A, Tilley N, et al. Host-to-host congestion control for TCP. IEEE COMST, 2010, 12(3): 304-342.
- [25] Jain R K, Chiu D M W, Hawe W R. A quantitative measure of fairness and discrimination. DEC Research Report, 1984.
- [26] Finn C, Abbeel P, Levine S. Model-agnostic meta-learning for fast adaptation of deep networks. In Proc. ICML, 2017: 1126-1135.